

Reconhecimento de Gestos/Ações com Esqueletos 2D e Transformers

Plano de Projeto e Andamento

Davi Baechtold

PUCPR

16 de setembro de 2025

Agenda

Objetivo

- ▶ Desenvolver um pipeline geral de reconhecimento de gestos/ações a partir de vídeos RGB monoculares via esqueletos 2D (mãos/pose) e Transformer.
- ▶ Foco abrangente (HCI, XR, robótica, esportes, segurança, educação) – não restrito a comunicação entre surdos e mudos.
- ▶ Entrega: código reprodutível, modelos treinados, demo em tempo real e documentação.

Escopo

- ▶ Extração de keypoints com MediaPipe (mãos/pose) e preparação de sequências temporais.
- ▶ Classificação com Transformer encoder; baselines LSTM/MLP para comparação.
- ▶ Treinamento, avaliação e demo em tempo real com webcam.
- ▶ Suporte a múltiplos datasets (ex.: Jester, EgoGesture, NTU RGB+D (2D), HaGRID, etc.).

Arquitetura do Pipeline

- ▶ **Entrada:** vídeo RGB ou sequência de frames.
- ▶ **Extração:** MediaPipe Hands/Pose $\Rightarrow$ keypoints 2D por frame.
- ▶ **Pré-processamento:** pad/trim para T quadros, normalização (z-score por amostra), máscara temporal.
- ▶ **Modelo:** projeção linear $\Rightarrow$ positional encoding $\Rightarrow$ Transformer encoder (multihead) $\Rightarrow$ pooling (CLS) $\Rightarrow$ cabeça MLP.
- ▶ **Saída:** logits $\Rightarrow$ classe de gesto/ação.

Implementação (Resumo)

- ▶ Código: `Project/src` (`data`, `models`, `utils`), scripts em `Project/scripts`.
- ▶ Configuração YAML: `Project/configs/transformer.yaml`.
- ▶ Treino: `train.py`; Avaliação: `evaluate.py`; Demo: `demo.py`.
- ▶ Teste de fumaça: `tests/test_smoke.py` (verifica formas do Transformer).

Datasets considerados

- ▶ **20BN-Jester** (dinâmico, mãos): leve, ótimo para Gestos manuais; já integrado.
- ▶ **EgoGesture** (dinâmico, egocêntrico): variedade de gestos, cenário natural; maior porte.
- ▶ **HaGRID** (estático, mãos): 2.8M imagens; útil para pré-treino de detector/classificador estático.
- ▶ **NTU RGB+D 60/120** (ações corporais): pode-se extrair 2D (MediaPipe) para usar no pipeline.
- ▶ **ChaLearn LAP IsoGD / Montalbano** (gestos): conjuntos multimodais (RGB-D), uso acadêmico.
- ▶ **Something-Something V2 / Kinetics** (ações gerais): muito grandes; seleção de subset recomendada.

Critérios de seleção

- ▶ Licença/uso acadêmico, tamanho e facilidade de download.
- ▶ Alinhamento com os gestos-alvo e com o escopo geral.
- ▶ Facilidade de integração: frames em diretórios ou vídeos; rótulos claros; divisão treino/val/teste.

Fase 1: Plano de Projeto (até 03/10)

- ▶ **Semana 1 (10/09–17/09):** Introdução + Referencial (draft); organizar base de vídeos; rodar `extract_keypoints.py`.
- ▶ **Semana 2 (18/09–24/09):** Metodologia (pipeline + arquitetura); iniciar treinamentos (Transformer/baselines).
- ▶ **Semana 3 (25/09–01/10):** Consolidar Plano de Projeto (26/09); preparar Resultados iniciais e slides.

Fase 2: Desenvolvimento (até 12/11)

- ▶ **Semanas 4–6 (02/10–22/10):** Refino e re-treino; ajustar hiperparâmetros; obter `best.pt`.
- ▶ **Semanas 7–8 (23/10–05/11):** Foco na demo (robustez e UX); Conclusão + revisão de referências.
- ▶ **Semana 9 (06/11–12/11):** Revisão final ABNT e entrega (código + documento) em 12/11.

Fase 3: Defesa Final (até 18/11)

- ▶ **Semana 10 (13/11–18/11):** Preparar slides da Defesa do Projeto Físico; ensaios e validação da demo.

Andamento

- ▶ Código funcional: extração, dataset, modelos, treino, avaliação e demo; teste de fumaça passa.
- ▶ Treino em CPU operando (fallback). **Limitação:** GPU atual (GTX 1050 Ti) insuficiente para treino rápido.
- ▶ Plano de mitigação: upgrade para **RTX 5060** e/ou uso de recursos da PUCPR (consultar laboratório/servidor).

Riscos principais

- ▶ **Hardware** (tempo de treinamento) $\Rightarrow$ Mitigar com RTX 5060 / servidor PUCPR; usar CPU apenas para validações rápidas.
- ▶ **Dados** (cobertura de classes) $\Rightarrow$ Ampliar com EgoGesture/NTU; balanceamento e curadoria de classes.
- ▶ **Agenda** (prazos curtos) $\Rightarrow$ Treinos incrementais com `-resume`; checkpoints frequentes; priorizar resultados mínimos viáveis.

Próximos Passos Imediatos

- ▶ Consolidar Metodologia no documento (pipeline, configs, parâmetros).
- ▶ Iniciar/retomar treinamentos com subsets e validação frequente; salvar métricas e confusão por época.
- ▶ Selecionar e baixar dataset adicional (EgoGesture/NTU subset) e integrar no manifest.
- ▶ Preparar slides finais da defesa do Plano de Projeto (com resultados parciais).

Resultados Parciais (val)

- ▶ Métricas (placeholder): Val (macro-F1): –% | Acc: –% | Loss: –
- ▶ Comparação (placeholder): LSTM vs MLP vs Transformer

Doc/Slides/cm_val.png

Resultados Finais (placeholder)

- ▶ Hiperparâmetros finais: `d_model=192`, `n_layers=4`, `n_heads=6`, `seq_len=64`, `batch=64` (ajustar se necessário).
- ▶ Teste (macro-F1): `-%` | Acurácia: `-%` | Tempo de inferência (CPU/GPU): `-/-ms`.
- ▶ Análise por classe: principais confusões, classes fáceis/difíceis.

Repositório e Execução

- ▶ Repositório: estrutura em `Project/` (código) e `Doc/` (texto/figuras).
- ▶ Execução (CPU): `CUDA_VISIBLE_DEVICES =`
`pythonProject/scripts/train.py --configProject/configs/transformer.yaml|Demo :`
`CUDA_VISIBLE_DEVICES = pythonProject/scripts/demo.py --checkpointbest.pt|`

Obrigado!
Perguntas?

Checklist e pontos para decidir

- ▶ Dataset adicional prioritário: EgoGesture vs NTU (subset) vs outro? Critérios de seleção.
- ▶ Acesso a recursos de treino: RTX 5060 local e/ou servidor PUCPR? Procedimentos e prazos.
- ▶ Escopo de classes/gestos alvo: taxonomia final (número de classes, definição por gesto/ação).
- ▶ Métricas de avaliação: macro-F1, acurácia, matriz de confusão, tempo de inferência (CPU/GPU).
- ▶ Demonstração: requisitos para a defesa (resolução, FPS, cenário de teste, roteiros).
- ▶ Riscos e mitigação: cronograma, backups de checkpoint, plano B para hardware/dados.